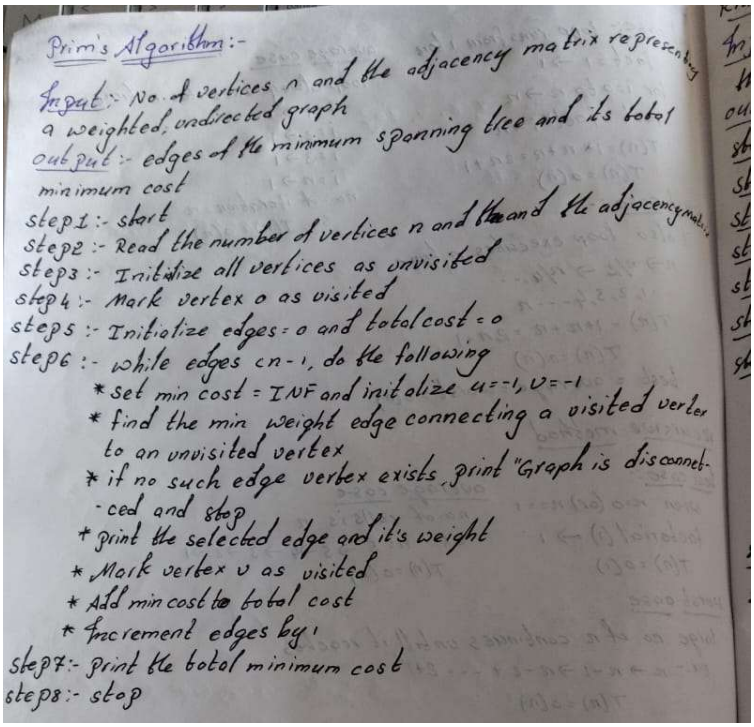
PRACTICAL: 09

AIM:- Implement prim's algorithm.

Algorithms:-



Program/code:-

```
#include <iostream>

using namespace std;

#define INF 9999

int main() {

int n;

cout << "Enter number of vertices: ";

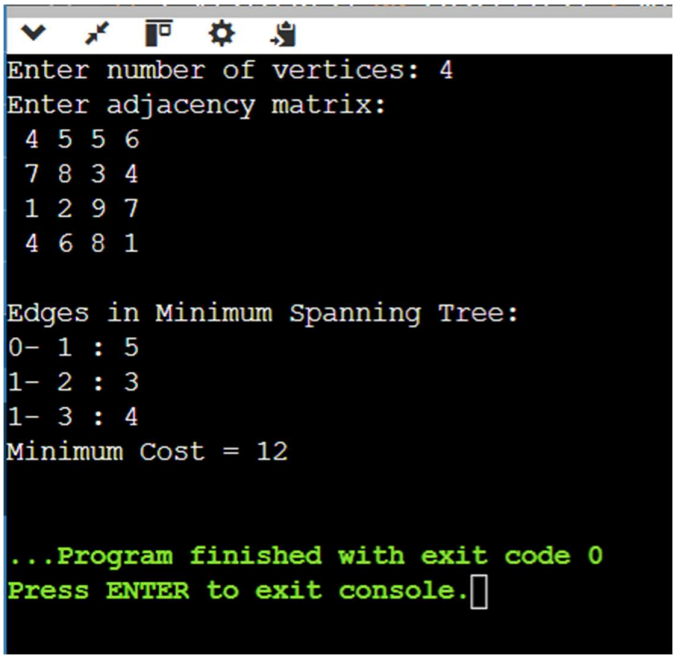
cin >> n;

int cost[10][10];
```

```
cout << "Enter adjacency matrix:\n";
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
cin >> cost[i][j];
if (cost[i][j] == 0)
cost[i][j] = INF;
}
}
int visited[10] = {0};
visited[0] = 1;
int edges = 0, totalCost = 0;
cout << "\nEdges in Minimum Spanning Tree:\n";
while (edges < n- 1) {
int minCost = INF;
int u =-1, v =-1;
for (int i = 0; i < n; i++) {
if (visited[i]) {
for (int j = 0; j < n; j++) {
if (!visited[j] && cost[i][j] < minCost) {
minCost = cost[i][j];
u =i;
v =j;
}
}
}
}
if (u ==-1) {
```

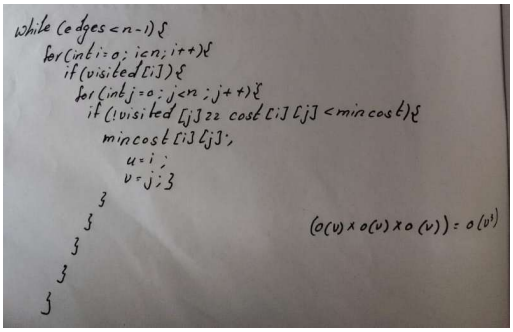
```
cout << "Graph is disconnected!\n";
return 0;
}
cout << u << "- " << v
<< " : " << minCost << endl;
visited[v] = 1;
totalCost += minCost;
edges++;
}
cout << "Minimum Cost = " << totalCost << endl;
return 0;
}
```

Output:-

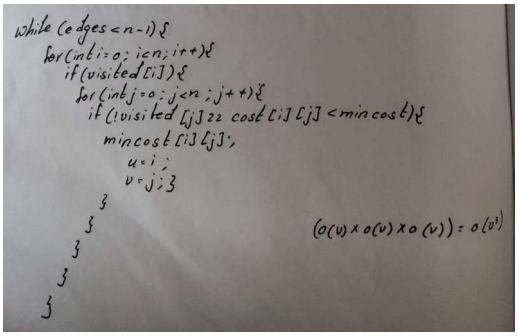


Time complexity:-

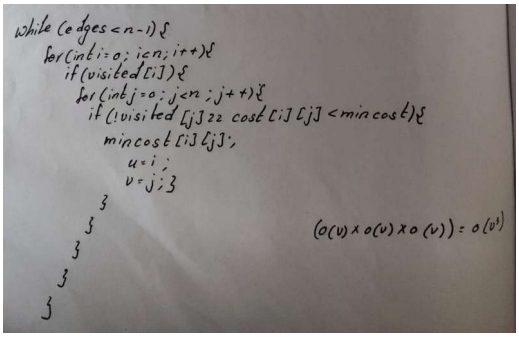
Best case:-



Worst case:-



Average case:-



Conclusion :-

An implementation of Prim's Algorithm in Python using a min-heap (priority queue) ensures an efficient time complexity of \(O(E \log V)\), where E is the number of edges and V is the number of vertices.